

멀티미디어공학

전공: 항공전자정보

이름: 김동현

학번: 2019124018

프로젝트에서 공통으로 쓰이는 코드(이미지 경로 함수, MSE 함수, plt 함수 등)은 26P 부록에 정리되어있습니다.

1. [그림1] 같은 16bit 깊이영상이 주어졌다고 가정하자. 16-bit 깊이 영상은 단일 채널로 구성되어 있으나, 일반적으로 사용되는 8bit 채널 시스템에서는 모든 정보를 전송할 수 없다. 이와 같은 높은 심도를 가지는 영상에 대한 표현 및 전송을 위한 방법으로, 아래와 같은 다양한 매핑 방법을 생각해 볼 수 있다.



[그림1]

[방법1] 16bit의 깊이영상에서 상위 8-bit(=256) 정보만을 단일 채널을 통해 전달하는 방법.

[방법2] 16bit의 정보 중 1536개의 정보만을 추출하고, 이를 RGB 3채널 공간 매핑 방식을 적용하는 방법.

[방법3] 깊이 영상을 상위 6bit, 중위 6bit, 하위 4-bit로 분리하여 매핑하는 방법.

A. 각각의 방법을 구현하고, 매핑된 정보를 display해 보아라.

[방법1]

접근: 비트 연산자를 이용하여 16비트 중 상위 8비트만 남긴다.

핵심코드:

```
image_16bit = cv2.imread(image_path, cv2.IMREAD_UNCHANGED)
upper_8bit = (image_16bit >> 8).astype(np.uint8)
```

코드설명: 16비트를 8비트로 변환하는 코드이다. cv2 함수로 이미지를 불러온 후에 비트 연산자를 이용하여 16비트 값을 8비트로 변환한다.

실행결과:

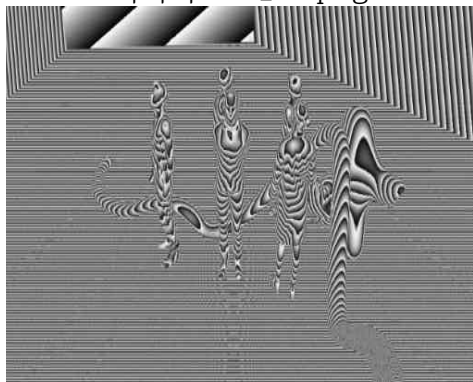
※ 추가로 하위 8비트 정보도 매핑하여 구현하였습니다.

원본 이미지

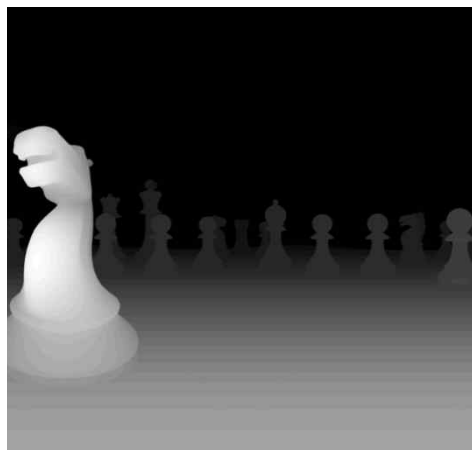
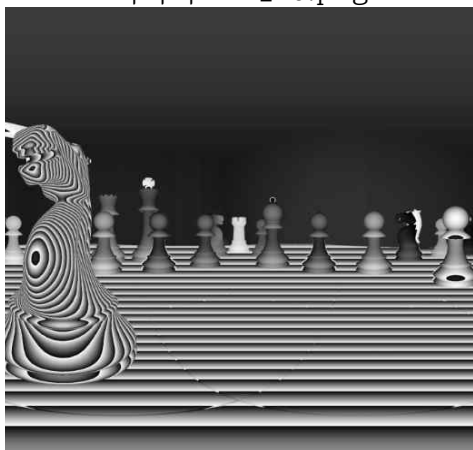
하위 8비트 이미지

상위 8비트 이미지

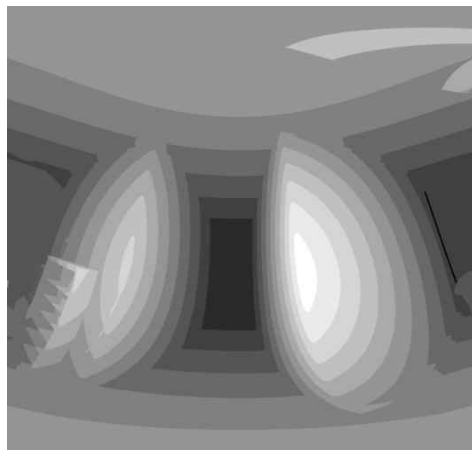
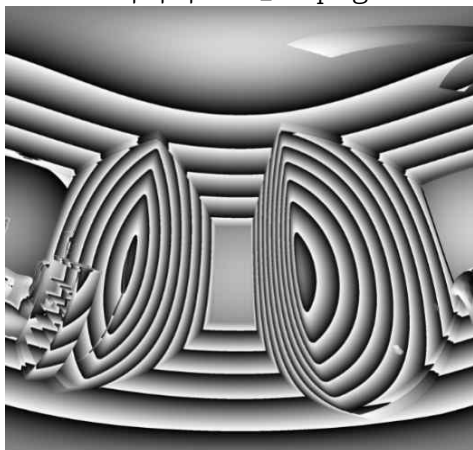
이미지: D1_16.png



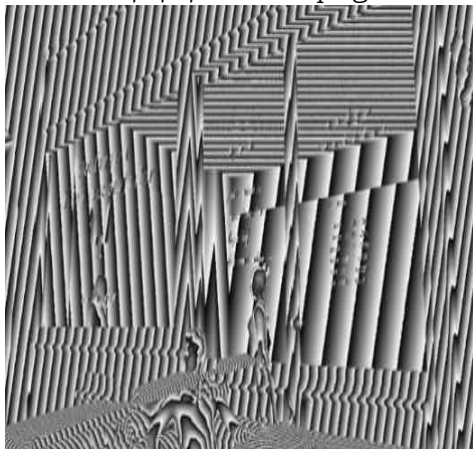
이미지: D2_16.png



이미지: D3_16.png



이미지: D4_16.png

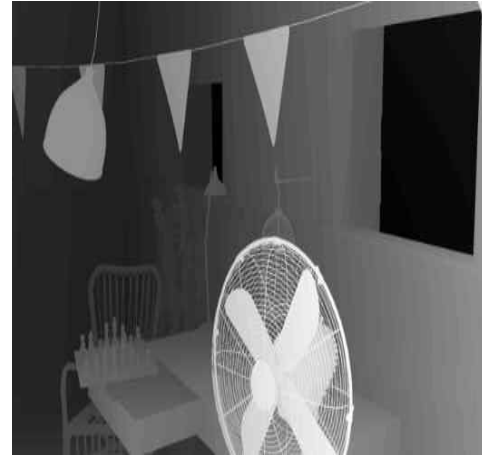
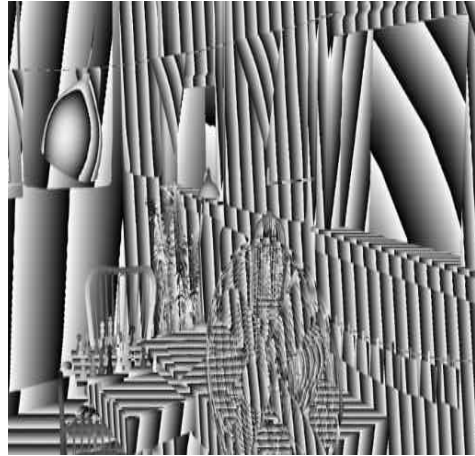
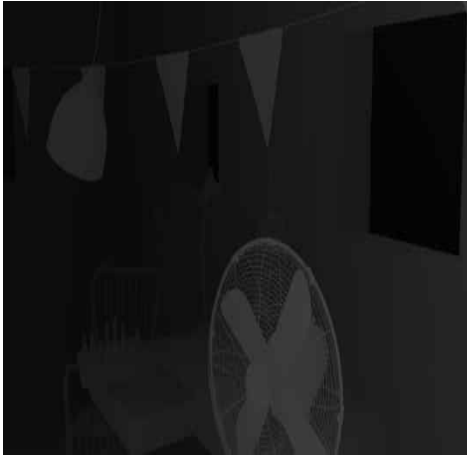


원본 이미지

하위 8비트 이미지

상위 8비트 이미지

이미지: D5_16.png



[방법2]

접근: 정규화를 통해 16비트(65536개) 정보 중 1536개의 정보만을 추출하고, 다음 6개의 구간에 0부터 1535를 256개씩 차례로 매핑한다. [구간1(255, 0~255, 0), 구간2(255~0, 255, 0), 구간3(0, 255, 0~255), 구간4(0, 255~0, 255), 구간5(0~255, 0, 255), 구간6(255, 0, 255~0)]

핵심코드:

```
normalized_image = (image / 65535 * 1535).astype(np.uint16)

def map_to_rgb(value):
    if 0 <= value < 256:
        return (255, value, 0)                                #(255, 0~255, 0)
    elif 256 <= value < 512:
        return (255 - (value - 256), 255, 0)                 #(255~0, 255, 0)
    elif 512 <= value < 768:
        return (0, 255, value - 512)                          #(0, 255, 0~255)
    elif 768 <= value < 1024:
        return (0, 255 - (value - 768), 255)                  #(0, 255~0, 255)
    elif 1024 <= value < 1280:
        return (value - 1024, 0, 255)                          #(0~255, 0, 255)
    elif 1280 <= value < 1536:
        return (255, 0, 255 - (value - 1280))                 #(255, 0, 255~0)

# 원본에서 RGB로 변환
for i in range(normalized_image.shape[0]):
    for j in range(normalized_image.shape[1]):
        rgb_image[i, j] = map_to_rgb(normalized_image[i, j])
```

코드설명: 정규화를 통해 16비트의 정보 중 1536개의 정보를 추출하고 (r,g,b)값을 할당하는 코드이다. map_to_rgb()라는 함수를 선언하였고 else if 구문을 통해 입력 값에 따른 출력 값을 반환하도록 하였다. 예를 들어 1026값을 입력 받으면 5번째 구문에 의하여 (2,0,255)를 반환한다. 마지막으로 for 구문을 통해 RGB이미지를 생성한다.

실행결과:

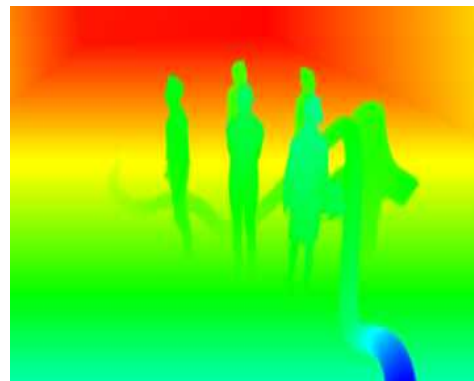
※ 추가로 1536개의 정보로 정규화한 이미지도 구현하였습니다.

원본 이미지

1536 정규화 이미지

RGB 이미지

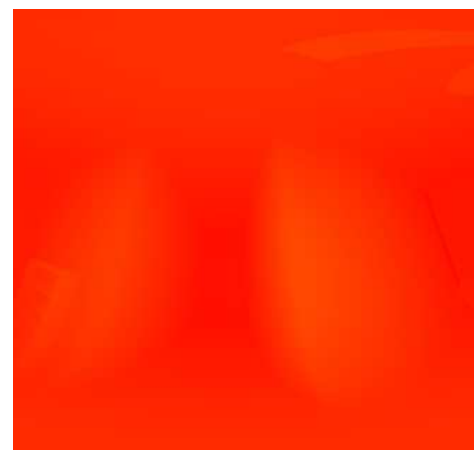
이미지: D1_16.png



이미지: D2_16.png



이미지: D3_16.png



이미지: D4_16.png



이미지: D5_16.png



분석: 작성한 함수에 의해 원본 이미지의 어두운 부분(크기가 0에 가까운 픽셀)은 RGB_이미지에서 빨간색이 강조되었고 회색(크기가 512~768사이인 픽셀)인 부분은 RGB이미지에서 초록색이 강조되었다. 1번 RGB_이미지 같은 경우 파란색이 강조된 부분이 있는 걸로 보아 해당 부분의 원본 이미지는 다른 원본 이미지에 비해 밝다는 것을 알 수 있다. 또한 육안으로 보아 1536으로 정규화한 이미지가 원본 이미지와 큰 차이가 없음을 알 수 있다. 이는 깊이 영상 전송 시, 용량 축소 부분에서 활용 될 수 있을 것으로 보인다.

[방법3]

접근: [방법1]과 유사하게 비트 연산자를 이용하여 16비트를 6,6,4 비트로 분할하여 각각 RGB 채널에 매핑한다.

핵심코드:

```
def map_16bit_to_rgb(image_16bit):
    height, width = image_16bit.shape
    rgb_image = np.zeros((height, width, 3), dtype=np.uint8)

    #16비트 이미지를 6비트, 6비트, 4비트로 분리 매핑
    for i in range(height):
        for j in range(width):
            pixel_value = image_16bit[i, j]

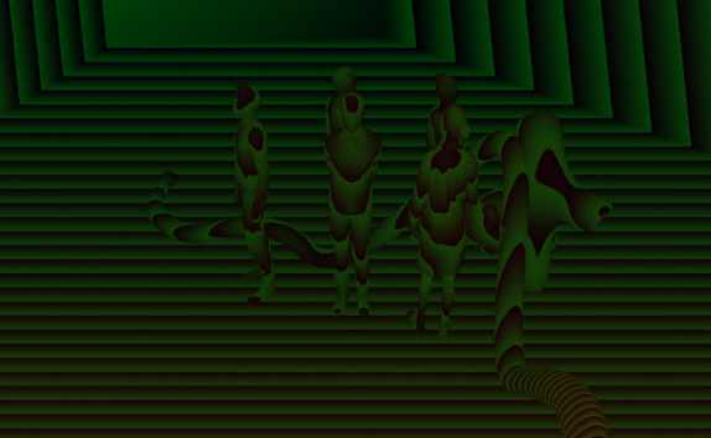
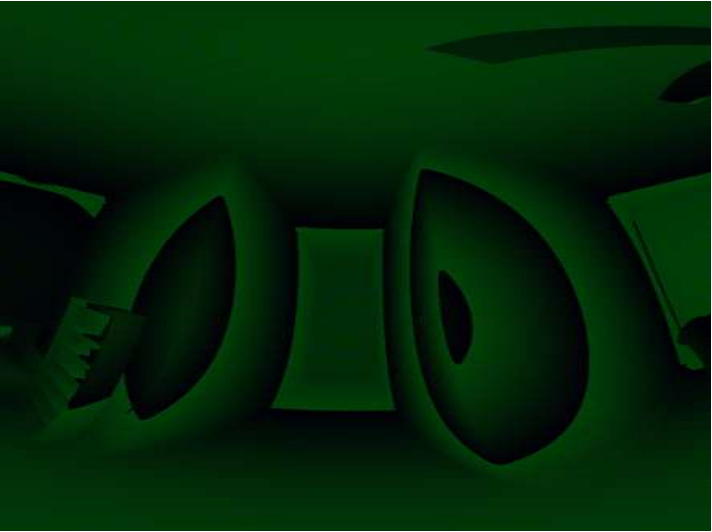
            r_ch = (pixel_value >> 10) & 0x3F    #상위 6비트
            g_ch = (pixel_value >> 4) & 0x3F    #중간 6비트
            b_ch = pixel_value & 0x0F          #하위 4비트

            rgb_image[i, j, 0] = r_ch          #R 채널
            rgb_image[i, j, 1] = g_ch          #G 채널
            rgb_image[i, j, 2] = b_ch          #B 채널

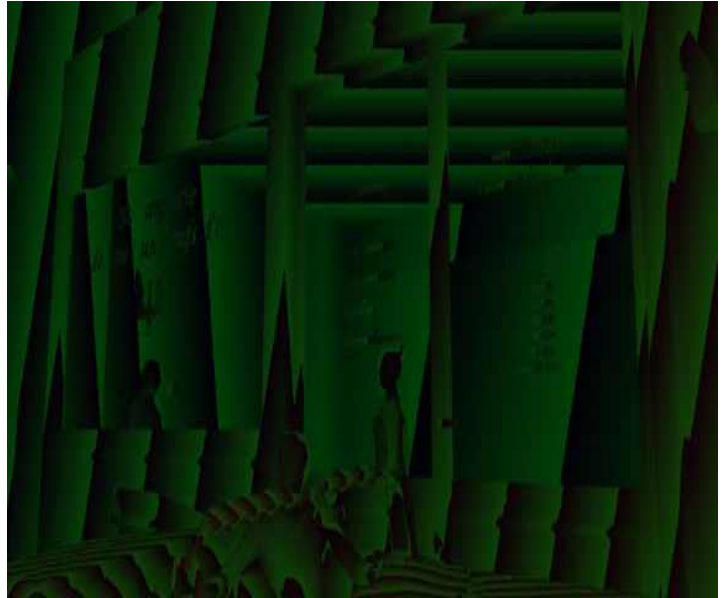
    return rgb_image
```

코드설명: 입력 이미지(원본 이미지)의 높이(height)와 너비(width)를 추출한다. 추출한 값을 이용하여 차원이 3인 RGB 이미지를 저장할 배열을 초기화한다. 그 후에 for 구문 안에서 픽셀을 매핑한다. 비트 연산자를 통해 16비트를 상위 6비트, 중간 6비트, 하위 4비트로 분리하고 RGB 채널에 각각 할당한다. 이때 하위비트부터 채운다.

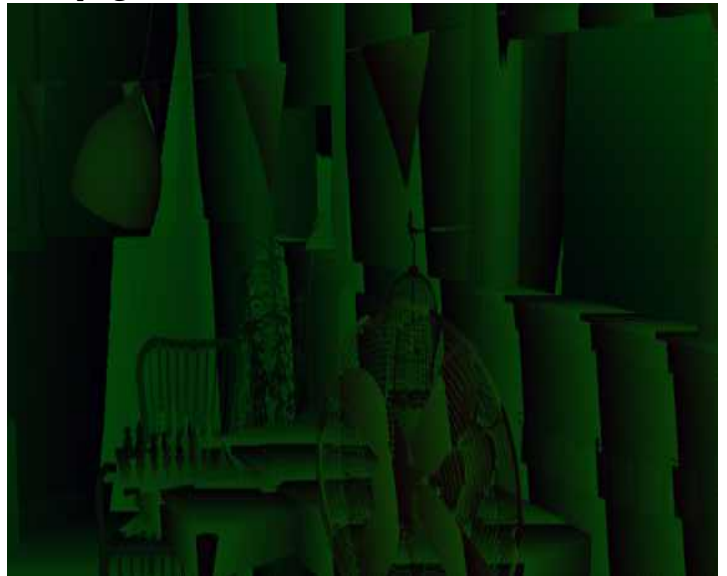
실행결과:

원본 이미지	RGB 이미지
이미지: D1_16.png	
	
이미지: D2_16.png	
	
이미지: D3_16.png	
	

이미지: D4_16.png



이미지: D5_16.png



분석: 변환된 RGB이미지는 원본 이미지에 비해 색상의 깊이와 그라데이션이 어느 정도 손실되었지만 색상 변화가 뚜렷하기 때문에 객체의 형태가 더 눈에 띄게 나타나는 장점이 있다.

B. 각각의 방법으로 매핑된 정보를 다시 16-bit 깊이 영상으로 복원하고, 복원 영상의 MSE를 계산하여라.

[방법1]

접근: 추출한 8비트 이미지를 다시 16비트로 복원한다. 이때 8비트 값을 왼쪽으로 시프트하여 16비트로 복원한다.

핵심코드: `restored_16bit = (upper_8bit.astype(np.uint16) << 8)`

코드설명: upper_8bit는 문제A의 [방법1]에서 추출한 8비트 값이다. 비트 연산자를 이용하여 16비트로 변환하는 코드이다.

실행결과:

	D1_16.png	D2_16.png	D3_16.png	D4_16.png	D5_16.png
MSE 값	21680.47	11861.11	21531.09	21649.76	21624.99

원본 이미지	복원된 이미지
--------	---------

이미지: D1_16.png, MSE: 21680.47



이미지: D2_16.png, MSE: 11861.11



이미지: D3_16.png, MSE: 21531.09



원본 이미지

복원된 이미지

이미지: D4_16.png, MSE: 21649.76



이미지: D5_16.png, MSE: 216424.99



분석: 복원된 이미지는 16비트 원본 이미지를 8비트로 축소했다가 다시 16비트 복원한 이미지이다. 원본 이미지와 복원된 이미지의 MSE 값은 2번째 이미지를 제외하고는 2만1천 정도가 나왔다. (2번째 이미지는 다른 이미지에 비해 어두워서(대부분 픽셀 값 자체가 0에 가까워서) 정보 손실 정도가 작은 것으로 예측된다.) 2만대의 MSE 값에도 불구하고 색 대비와 밝기 등을 고려해볼 때 시각적인 차이는 크게 두드러지지 않는다. 상위 8비트 정보만을 사용하여 복원하는 과정에서 객체의 구조나 윤곽이 대부분 보존되는 것으로 판단된다.

[방법2]

접근: RGB 값을 0과 1535사이의 값으로 복원한 후에 16비트 크기로 맞춰주기 위해 스케일링을 한다.

핵심코드:

```
def map_to_value(r, g, b):
    if r == 255 and 0 <= g < 256 and b == 0:
        return g
    elif 0 <= r < 256 and g == 255 and b == 0:
        return 256 + (255 - r)
    elif r == 0 and g == 255 and 0 <= b < 256:
        return 512 + b
    elif r == 0 and 0 <= g < 256 and b == 255:
        return 768 + (255 - g)
    elif 0 <= r < 256 and g == 0 and b == 255:
        return 1024 + r
    elif r == 255 and g == 0 and 0 <= b < 256:
        return 1280 + (255 - b)

for i in range(rgb_image.shape[0]):
    for j in range(rgb_image.shape[1]):
        r, g, b = rgb_image[i, j]
        restored_image[i, j] = map_to_value(r, g, b)
restored_image = (restored_image / 1535 * 65535).astype(np.uint16)
```

코드설명: 함수 map_to_value(r, g, b)는 RGB 값을 입력 받아, 해당하는 값을 특정 범위(0~1535)로 변환하는 역할을 한다. 마지막 for구문을 통해 함수를 실행하고 restored image에 변환된 값들을 저장한다. 최종적으로 16비트 크기로 맞춰주기 위해 1535로 나눈 후, 65535를 곱하여 확장시켜준다.

실행결과:

	D1_16.png	D2_16.png	D3_16.png	D4_16.png	D5_16.png
MSE 값	640.15	613.59	639.10	635.34	626.85

원본 이미지	복원된 이미지
--------	---------

이미지: D1_16.png, MSE: 640.15



원본 이미지

복원된 이미지

이미지: D2_16.png, MSE: 613.59



이미지: D3_16.png, MSE: 639.10



이미지: D4_16.png, MSE: 635.34



원본 이미지

복원된 이미지

이미지: D5_16.png, MSE: 626.85



분석: [방법1]에 비해 상당히 낮은 MSE 값을 갖는다. 스케일링 과정에서 이미지의 값의 범위가 정규화를 통해 1535로 줄었으며 복원 과정에서 map_to_rgb와 map_to_value 함수는 상호 간에 손실 없이 매핑을 수행하기 때문에 정보 손실이 비교적 적은 것으로 판단된다.

[방법3]

접근: 각각의 RGB 값을 비트 이동을 통해 R은 상위 4비트 G는 중간 6비트 B는 하위 4비트로 하여 결합하여 총 16비트로 복원한다.

핵심코드:

```
def map_rgb_to_16bit(rgb_image):
    height, width, _ = rgb_image.shape
    restored_image = np.zeros((height, width), dtype=np.uint16)

    for i in range(height):
        for j in range(width):
            r_ch = rgb_image[i, j, 0]
            g_ch = rgb_image[i, j, 1]
            b_ch = rgb_image[i, j, 2]

            restored_image[i, j] = (r_ch << 10) | (g_ch << 4) | b_ch

    return restored_image
```

코드설명: 함수 `map_rgb_to_16bit`는 RGB이미지를 입력받고 16비트 깊이 이미지로 복원하여 출력하는 함수이다. 3개의 채널 RGB의 값을 포함하는 `rgb_image`를 입력 받고 이중 반목문을 통해 각 픽셀의 RGB 값을 추출하여 각각 `r_ch`, `g_ch`, `b_ch`에 저장한다. 이때 `r_ch` 값은 10비트 왼쪽으로, `g_ch` 값은 4비트 왼쪽으로 이동시킨 후 세 값을 결합되며, 결합된 값은 `rgb_image`와 `height`와 `width`가 같고 모든 성분이 0이 `restored_image`로 매핑된다.

실행결과:

	D1_16.png	D2_16.png	D3_16.png	D4_16.png	D5_16.png
MSE 값	0	0	0	0	0

원본 이미지	복원된 이미지
--------	---------

이미지: D1_16.png, MSE: 0



이미지: D2_16.png, MSE: 0



이미지: D3_16.png, MSE: 0



이미지: D4_16.png, MSE: 0



이미지: D5_16.png, MSE: 0



분석: [방법3] 모든 이미지의 MSE 값이 0이 나온다. 다른 방법처럼 특정 정보만 추출하거나 정규화를 거치지 않았기 때문에 16비트 정보를 RGB로 변환하거나 그 반대 과정에서 정보가 손실되지 않았다. MSE 값만 고려하였을 때 3가지 방법 중 가장 우수한 방법으로 간주된다. 특히 [방법2]와 비교했을 때 똑같이 3채널을 쓰지만 MSE 값이 더 작으며 연산속도도 1분16초 더 빨랐다([방법2]의 연산속도는 4분17초이다).

문제B 최종분석: 아래 표는 위에서 수행한 이미지별 모든 방법의 MSE 값을 정리해놓은 표이다. MSE 값은 [방법1]이 가장 크며, 이미지 중에서는 D1-16 이미지의 MSE 값이 가장 크고 D2_16 이미지의 MSE 값이 가장 작다는 것을 알 수 있다. 이를 통해 비교적 명암 대비가 분명한 이미지일수록 복원 과정에서 손실되는 양이 더 많다는 것을 유추해볼 수 있다.

빨강: 최댓값 파랑: 최솟값

	D1_16.png	D2_16.png	D3_16.png	D4_16.png	D5_16.png
[방법1] MSE	21680.47	11861.11	21531.09	21649.76	21624.99
[방법2] MSE	640.15	613.59	639.10	635.34	626.85
[방법3] MSE	0	0	0	0	0

2. 문제 1의 복원 과정에서 채널 전송 시 오류가 발생하는 경우, 각각의 매핑 방식의 장단점을 비교해 보고자 한다.

A. 방법 1~3의 매핑 과정에서 아래와 같은 2가지 정류의 채널전송오류가 발생한다고 가정한다. 전송 오류는 컬러의 모든 공간에 동일하게 발생한다. 아래의 채널전송오류가 발생하는 상황에서 복원된 깊이 영상의 MSE를 각각 계산하여라.

- i . Gaussian noise with mean = 0, std = 10, 30, 50
- ii . JPEG compression noise with QP = 90, 60, 30

※ 모든 방법의 접근방식과 노이즈코드는 동일하므로, 이에 대한 설명은 바로아래 한 번만 작성하였습니다.

※ 또한 매핑 및 복원 코드는 [방법1], [방법2], [방법3] 각각 직전 문제와 동일하여 생략하였습니다.

접근방식: 매핑된 값에 가우시안 및 JPEG 압축 노이즈를 발생시킨 후 복원한다. 매핑 방식과 복원 방식은 직전문제(문제1의 B)와 동일하며 코드 역시 그대로 사용한다. 노이즈코드의 위치는 복원하는 과정 바로 앞으로 한다. 이러한 접근 방식과 노이즈 발생 코드를 [방법1]~[방법3]에 모두 적용한다.

노이즈코드:

```
# 1. Gaussian 노이즈 추가
noise = np.random.normal(평균, 표준편차, rgb_image.shape).astype(np.int16)
noisy_rgb_image = np.clip(rgb_image.astype(np.int16) + noise, 0, 255).astype(np.uint8)

# 2. JPEG 압축 노이즈 추가
encode_param = [int(cv2.IMWRITE_JPEG_QUALITY), QP]
_, compressed_img = cv2.imencode('.jpg', rgb_image, encode_param)
decoded_img = cv2.imdecode(compressed_img, cv2.IMREAD_UNCHANGED)
```

코드설명: 함수 `np.random.normal`는 주어진 평균과 표준편차를 기준으로 가우시안 분포를 따르는 난수를 생성하며, 생성된 난수의 모양은 매핑된 RGB 이미지와 동일하다. RGB 이미지에 생성된 노이즈(난수)를 더하여 `noisy_rgb_image`를 형성한다. (`np.clip`은 픽셀 값이 8비트 범위를 유지하도록 클리핑 동작을 수행한다.)

`encode_param`는 JPEG 압축 품질을 설정하는 파라미터이다. QP는 JPEG 품질 인덱스를 의미한다. `cv2.imencode()`는 RGB 이미지를 JPEG 형식으로 압축한다. `cv2.imdecode()`는 반대로 복원하는 동작을 하며 복원된 이미지는 `decoded_img`에 저장한다.

[방법1]

실행결과:

■: MSE 최대 ■: MSE 최소

[방법1] MSE			이미지 종류				
			D1_16.png	D2_16.png	D3_16.png	D4_16.png	D5_16.png
노이즈 종류	가우시안 노이즈	std = 10	25237.19	23201.33	25129.56	25176.76	25129.95
		std = 30	32094.70	25078.29	32181.96	32190.50	32157.92
		std = 50	32227.05	25183.62	32379.40	32368.56	32310.84
	J P E G 압축 노이즈	QP = 90	20675.75	11870.67	21372.04	21663.29	22377.38
		QP = 60	20483.79	13240.15	23534.00	21935.96	22234.16
		QP = 30	25004.45	14607.29	21997.70	25097.62	26239.86
노이즈 없음			21680.47	11861.11	21531.09	21649.76	21624.99

이미지: MSE 값이 대체로 큰 D5_16.png 이미지를 시각화하였다.

원본

std = 10, MSE = 25129.95



std = 30, MSE = 32157.92

std = 50, MSE = 32310.84



원본 확대 버전

QP = 90 (이미지 확대 버전), MSE = 22377.38



QP = 60 (이미지 확대 버전), MSE = 22234.16

QP = 30 (이미지 확대 버전), MSE = 26239.86



[방법2]

추가된 코드: 기존 함수에 다음과 같은 코드를 추가하여 RGB이미지에서 원본 이미지로 복원할 때, 노이즈가 발생한 픽셀이 조건문에서 해당하는 조건이 없을 경우 0으로 보정하여 오류를 방지한다.

```
def map_to_value(r, g, b):
    # 기존 조건들
    if r == 255 and 0 <= g < 256 and b == 0:
        return g
    elif 0 <= r < 256 and g == 255 and b == 0:
        return 256 + (255 - r)
    elif r == 0 and g == 255 and 0 <= b < 256:
        return 512 + b
    elif r == 0 and 0 <= g < 256 and b == 255:
        return 768 + (255 - g)
    elif 0 <= r < 256 and g == 0 and b == 255:
        return 1024 + r
    elif r == 255 and g == 0 and 0 <= b < 256:
        return 1280 + (255 - b)
    # 추가된 조건
    else :
        return 0
```

실행결과:

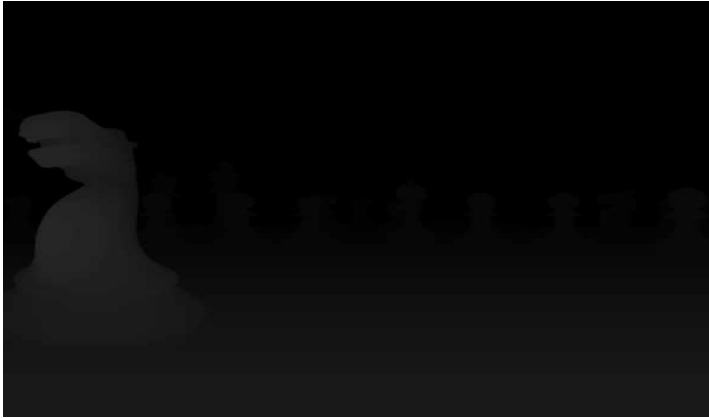
■ : MSE 최대 ■ : MSE 최소

[방법2] MSE			이미지 종류				
			D1_16.png	D2_16.png	D3_16.png	D4_16.png	D5_16.png
노이즈 종류	가우시안 노이즈	std = 10	30212.38	20114.26	30551.52	30419.07	30388.65
		std = 30	31754.40	21292.07	32112.76	32002.24	31936.87
		std = 50	32033.89	21501.89	32384.28	32255.95	32217.00
	J P E G 압축 노이즈	QP = 90	14881.74	9189.49	14698.38	16768.47	17965.74
		QP = 60	21898.32	12909.37	19792.88	22921.94	23012.60
		QP = 30	27472.03	14740.91	29147.23	26741.99	27183.23
노이즈 없음			640.15	613.59	639.10	635.34	626.85

이미지: D2_16.png와 D5_16.png를 각각 가우시안 노이즈, JPEG 압축 노이즈 상황에서 복원하였다.

원본 (D2_16.png)

std = 10, MSE = 20114



std = 30, MSE = 21292.07

std = 50, MSE = 21501.89



원본 D5_16.png

QP = 90, MSE = 17965.74



QP = 60, MSE = 23012.60

QP = 30, MSE = 27183.23



[방법3]

실행결과:

■: MSE 최대 ■: MSE 최소

[방법3] MSE			이미지 종류				
			D1_16.png	D2_16.png	D3_16.png	D4_16.png	D5_16.png
노이즈 종류	가우시안 노이즈	std = 10	29695.87	24776.33	29640.33	29592.36	30142.65
		std = 30	31901.12	27511.69	31731.31	31878.31	32083.53
		std = 50	32161.56	27982.30	31667.76	32054.96	32194.95
	J P E G 압축 노이즈	QP = 90	24318.99	14334.48	21073.11	23716.34	25168.92
		QP = 60	26692.95	17284.78	22375.78	25279.31	25978.28
		QP = 30	28064.61	18213.60	27307.49	26992.60	28404.64
노이즈 없음			0	0	0	0	0

이미지: MSE 값이 대체로 큰 D5_16.png 이미지를 시각화하였다.

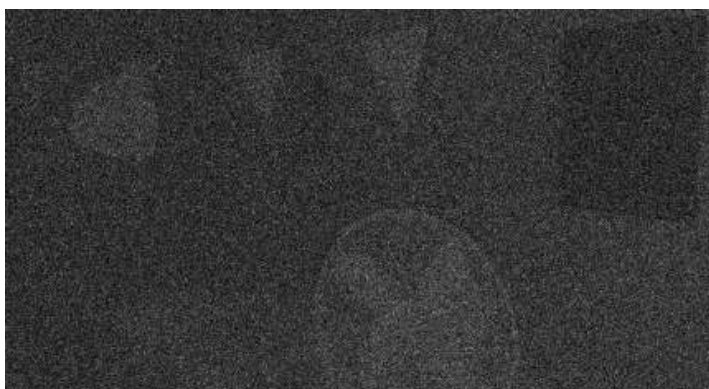
원본

std = 10, MSE = 30142.65



std = 30, MSE = 32083.53

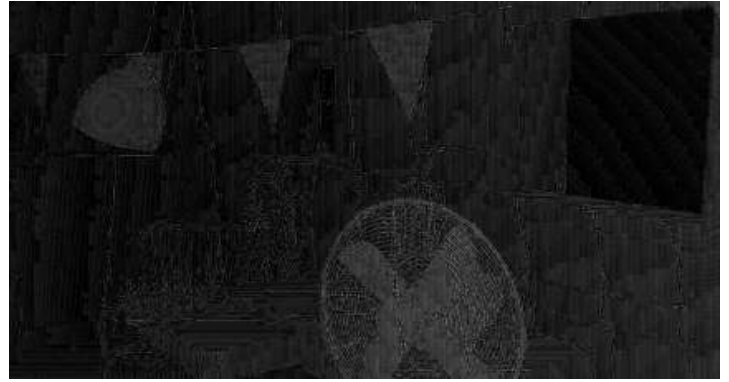
std = 50, MSE = 32194.95



QP = 90, MSE = 26168.92



QP = 60, MSE = 25978.28



QP = 30, MSE = 28404.64



문제A 최종분석: (<표1> 참고) 3가지 방법 모두 노이즈가 없을 때보다 가우시안 노이즈가 발생하였을 때 MSE가 더 높게 나오며, 표준편차가 커질수록 MSE 값도 더 커졌다. 이는 가우시안 노이즈의 특성으로 표준편차가 커질수록 값이 큰 노이즈의 개수가 증가하기 때문이다. JPEG 압축 노이즈도 [방법2]와 [방법3]은 MSE의 경향이 가우시안 노이즈 상황과 똑같다. QP가 높을수록 정보가 더 적게 손실되므로 MSE도 더 낮게 나온다. 이와 달리 [방법1]은 이미지의 종류와 QP에 따라 MSE가 노이즈 없을 때보다 더 낮게 나올 때가 있고, 심지어 QP가 높아지는데 반대로 MSE가 커지는 경우가 생긴다. 첫 번째 현상의 원인은 [방법1]의 매핑 방식에 있다. 매핑 과정에서 원본 데이터의 하위 8비트를 0으로 만드는 손실이 발생하는데, 오히려 노이즈가 이 공간을 채우면서 어느정도 보정되는 효과가 있기 때문이다. 두 번째 현상은 이미지 특성에 의한 것으로 판단된다. 매핑된 (상위 8비트만 추출된) 이미지 D3_16.png는 경계선이 부드럽고 텍스처가 단순한데, 이러한 특성 때문에 높은 QP로 압축할 경우 경계선과 텍스처에서 약간의 압축 왜곡이 발생할 수 있다. 오히려 경계선과 텍스처가 단순할수록 QP가 낮아도 손실된 부분이 원본과 크게 다르지 않을 수 있다. 실제로 MSE는 QP=30 일 때가 QP=60 일 때보다 낮다.

<표 1>

[방법1] MSE		이미지 종류				
		D1_16.png	D2_16.png	D3_16.png	D4_16.png	D5_16.png
J P E G 압 축 노 이즈	QP = 90	20675.75	11870.67	21372.04	21663.29	22377.38
	QP = 60	20483.79	13240.15	23534.00	21935.96	22234.16
	QP = 30	25004.45	14607.29	21997.70	25097.62	26239.86
노이즈 없음		21680.47	11861.11	21531.09	21649.76	21624.99

B. 방법 1~3의 매핑 방법을 참고하여, 채널의 전송오류 상황에 강인한 최적의 매핑 방법을 제안하고 결과를 검증하여라.

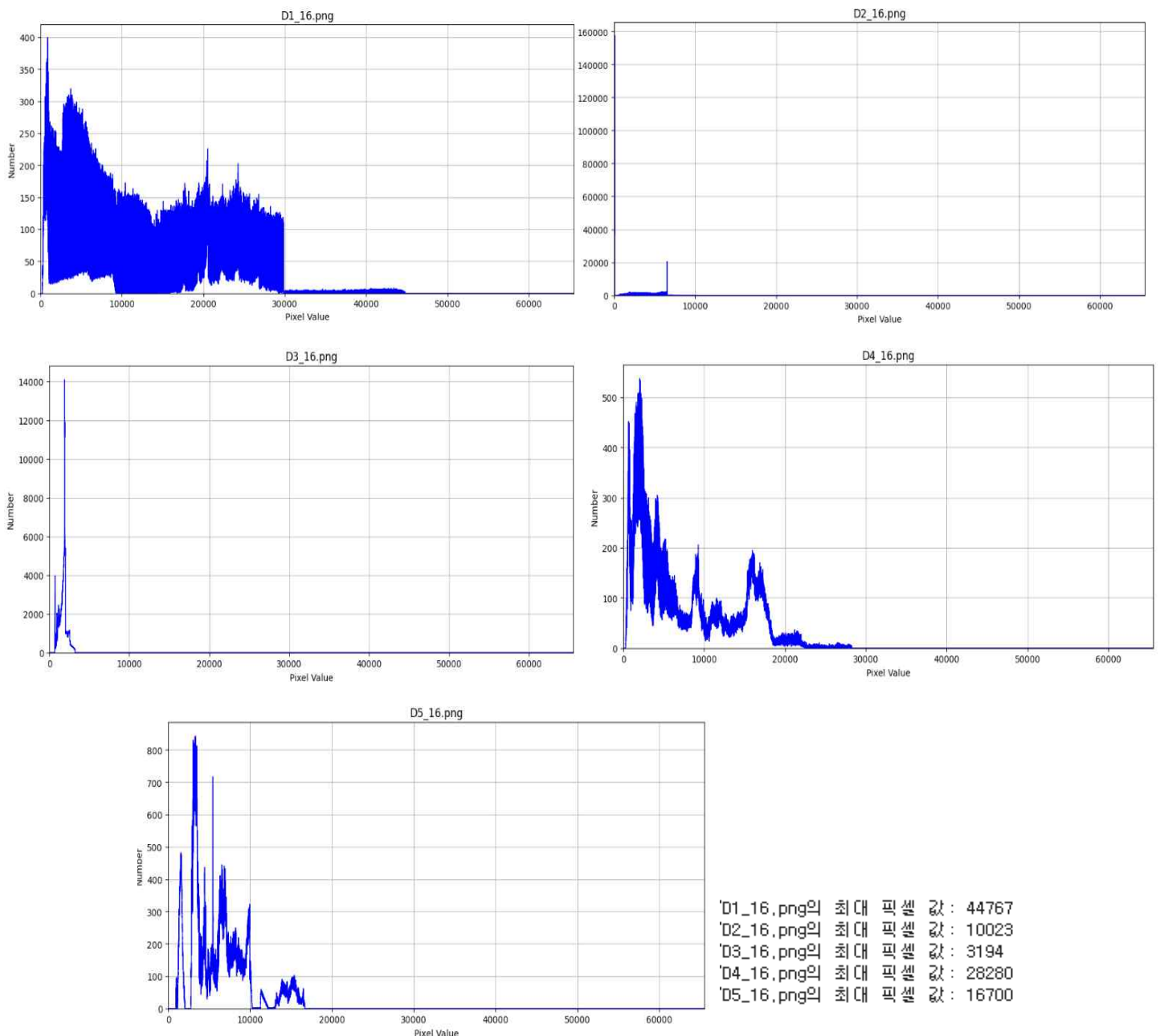
주어진 조건:

- i. 매핑 과정에서 사용할 수 있는 공간은 최대 24bit이다.(RGB 3채널)
- ii. 복원 과정에서 발생하는 전송 오류는 가우시안 노이즈와 JPEG 압축 노이즈로 한정한다.

MSE를 줄이기 위해 고려할 사항:

- i. 1번 문제를 통해 매핑 정보가 많을수록 MSE 감소한다는 사실을 안다.(즉, [방법1]은 지양한다.)
- ii. 노이즈의 특성 파악(2번 B문제 마지막 분석 참고). ex) 노이즈 값 분포
- iii. 이미지 픽셀 값 분포를 확인하고 더 많은 비트를 할당한 구간을 고려한다.

`plt.hist(depth_image.ravel(), bins=65536, range=(0, 65535), color='blue', histtype='step')`로 확인



매핑 방식 설계:

- i. [방법2] 개선: 기존에는 구간별 선형 매핑을 하였다면, 이미지별 픽셀 값 분포를 확인하고 픽셀 값이 특정 구간에 몰려있다면 비선형 매핑을 고려해본다. 로그함수로 매핑하면 MSE를 낮추 수 있을 것으로 보인다. 또한 [방법2]는 매핑에 사용되지 않은 두 채널의 값은 각각 0, 255가 되는데 복원하는 과정에서 (r,g,b) 중 최솟값은 0으로 최댓값은 255로 보정하여 개선한다.
- ii. [방법3] 개선: 채널 별 비트 분배 비율을 기존과 다르게 하고, 채널의 상위 비트부터 할당하여 MSE를 낮출 수 있다. 또한 비트 분배 전 데이터를 정규화하여 노이즈 저항성을 증가시킬 수 있다.
- iii. 최종: 픽셀 분포를 통해 모든 이미지의 픽셀 최댓값이 44767임을 확인하였고, 0~44767값을 1535로 정규화 한 후에 RGB채널에 구간별 매핑을 한다. 또한 복원 과정에서 (r,g,b) 중 최솟값은 0으로 최댓값은 255로 보정하여 개선한다.

핵심 코드 및 설명:

```
#이미지를 불러오고 이미지의 최대 픽셀 값을 찾는 코드입니다
depth_image = cv2.imread(image_path, cv2.IMREAD_UNCHANGED)
max_pixel_value = np.max(depth_image)
```

```
#[방법2] 코드에 (r,g,b) 보정 코드를 추가하였습니다
def map_to_value(r, g, b):
    #추가된 코드
    min_val = min(r, g, b)
    max_val = max(r, g, b)
    if r == max_val: r = 255
    elif r == min_val: r = 0
    if g == max_val: g = 255
    elif g == min_val: g = 0
    if b == max_val: b = 255
    elif b == min_val: b = 0

    # 기존 조건들
    if r == 255 and 0 <= g < 256 and b == 0:
        return g
    elif 0 <= r < 256 and g == 255 and b == 0:
        return 256 + (255 - r)
    (...)
```

```
#[방법2]와 다르게 44767을 1535로 정규화 하였습니다.
normalized_image = (image /44767 * 1535).astype(np.uint16)
```

실행 결과: ※ 비교를 위해 [방법1], [방법2], [방법3]의 MSE 표를 첨부하였습니다.

■: MSE 최대 ■: MSE 최소

[최적의 방법] MSE			이미지 종류				
			D1_16.png	D2_16.png	D3_16.png	D4_16.png	D5_16.png
노이즈 종류	가우시안 노이즈	std = 10	22285.07	21396.58	22242.17	22243.44	22324.40
		std = 30	29326.17	26783.22	29292.88	29341.16	29363.73
		std = 50	30756.59	27874.59	30836.94	30854.18	30827.09
	J P E G	QP = 90	1645.00	1477.02	388.86	3014.00	4562.08
		QP = 60	3184.55	2232.67	1270.30	4907.06	6348.13
		QP = 30	6295.95	2172.01	2266.58	7153.78	7963.41

[방법1] MSE			이미지 종류				
			D1_16.png	D2_16.png	D3_16.png	D4_16.png	D5_16.png
노이즈 종류	가우시안	std = 10	25237.19	23201.33	25129.56	25176.76	25129.95
		std = 30	32094.70	25078.29	32181.96	32190.50	32157.92
		std = 50	32227.05	25183.62	32379.40	32368.56	32310.84
	J P E G	QP = 90	20675.75	11870.67	21372.04	21663.29	22377.38
		QP = 60	20483.79	13240.15	23534.00	21935.96	22234.16
		QP = 30	25004.45	14607.29	21997.70	25097.62	26239.86

[방법2] MSE			이미지 종류				
			D1_16.png	D2_16.png	D3_16.png	D4_16.png	D5_16.png
노이즈 종류	가우시안	std = 10	30212.38	20114.26	30551.52	30419.07	30388.65
		std = 30	31754.40	21292.07	32112.76	32002.24	31936.87
		std = 50	32033.89	21501.89	32384.28	32255.95	32217.00
	J P E G	QP = 90	14881.74	9189.49	14698.38	16768.47	17965.74
		QP = 60	21898.32	12909.37	19792.88	22921.94	23012.60
		QP = 30	27472.03	14740.91	29147.23	26741.99	27183.23

[방법3] MSE			이미지 종류				
			D1_16.png	D2_16.png	D3_16.png	D4_16.png	D5_16.png
노이즈 종류	가우시안	std = 10	29695.87	24776.33	29640.33	29592.36	30142.65
		std = 30	31901.12	27511.69	31731.31	31878.31	32083.53
		std = 50	32161.56	27982.30	31667.76	32054.96	32194.95
	J P E G	QP = 90	24318.99	14334.48	21073.11	23716.34	25168.92
		QP = 60	26692.95	17284.78	22375.78	25279.31	25978.28
		QP = 30	28064.61	18213.60	27307.49	26992.60	28404.64

이미지: D4_16.png 이미지를 시각화하였다.

원본

QP = 90, MSE = 3014.00



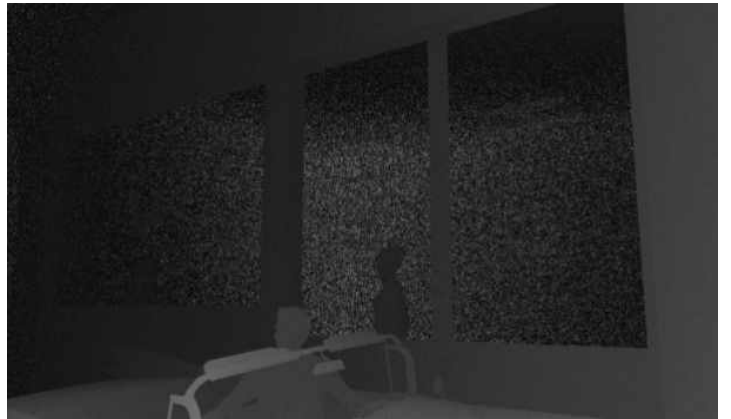
QP = 30, MSE = 7153.78

QP = 60, MSE = 4907.06



std = 10, MSE = 22243.44

std = 50, MSE = 30548.18



분석: 설계한 방법의 MSE가 기존 방법 대비 모든 경우에 대해 감소했음을 알 수 있다. 특히 JPEG 압축 노이즈에 대해서는 QP가 90일 때 MSE가 400까지 낮아진다. 이를 바탕으로 설계한 매핑 방식이 채널의 전송 오류 상황에 강인한 최적의 매핑 방법이라고 볼 수 있다.

아쉬운 점: 픽셀 값 분포를 고려해볼 때 이미지 D2, D3, D5는 비선형(로그함수) 매핑을 할 경우 효과가 좋을 것으로 예상되나 이미지 D1, D4가 부적합하여 사용하지 못하였다.

<부록>

이 페이지에는 프로젝트에서 공통으로 사용되는 코드에 대한 설명이 작성되어있습니다.

프로젝트에 사용할 모듈을 선언하고 이미지를 불러오는 코드입니다.

```
from google.colab import drive
drive.mount('/content/drive')
import numpy as np
import matplotlib.pyplot as plt
import cv2
import os
```

```
image_paths = [
    '/content/drive/My Drive/Colab Notebooks/D1_16.png',
    '/content/drive/My Drive/Colab Notebooks/D2_16.png',
    '/content/drive/My Drive/Colab Notebooks/D3_16.png',
    '/content/drive/My Drive/Colab Notebooks/D4_16.png',
    '/content/drive/My Drive/Colab Notebooks/D5_16.png'
]
```

원본이미지와 복원이미지의 MSE를 계산하는 함수입니다.

```
def calculate_mse(original, restored):
    return np.mean((original - restored) ** 2)
```

시각화를 위한 코드입니다.

```
plt.figure(figsize=(10, 20))

for idx, (restored_image, mse) in enumerate(results):

    plt.subplot( 행, 열, idx + 1)
    plt.title('이미지 제목')
    plt.imshow(restored_image, cmap='gray', vmin=최솟값, vmax=최댓값)
    plt.axis('off')

plt.show()
```